COMP.8157 ADVANCED DATABASE TOPICS | GROUP PROJECT DELIVERABLE D.4.1

Project Documentation: User Requirements and Analysis Document

A Change Stream Ledger for MongoDB: Tamper Evidence Without a Ledger Engine

Course	COMP.8157 Advanced Database Topics, Fall 2026, University of Windsor. Instructor: Dr. Andreas S. Maniatis
Group	Team # 3 (Group Category: Project)
Members	Olumide Ogunjebi (ogunjeb@uwindsor.ca), Rohith Nelluri (nellurir@uwindsor.ca), Yadu Pratap (Singh8a4@uwindsor.ca)
Document	23 October 2026. Status: submitted for review
Code repository	GitHub: paste the GitHub repository link here (in progress)
Project management	Hive: Open Hive project

Table of Contents

1. Introduction and Scope 3
1.1 Purpose of this document 3
1.2 Problem statement 3
1.3 Objectives 3
1.4 Scope and boundaries 4
1.5 Definitions 4
1.6 Assumptions and constraints 4
2. Stakeholder Roles 4
3. Analysis 5
3.1 Current situation and gap 5
3.2 Threat analysis 5
3.3 User stories 5
3.4 Prioritization and feasibility 6
4. Functional Requirements 6
4.1 Change capture 6
4.2 Sealing, publication and recovery 6
4.3 Verification 7
4.4 Search and logs 7
4.5 Evaluation tooling 7
5. Nonfunctional Requirements 7
6. Acceptance Criteria 8
7. Revision History 9
References 10

1. Introduction and Scope

1.1 Purpose of this document

This document records what the system must do, for whom, and how the team will prove that each requirement has been met. It turns the approved full proposal (D.2.1) into numbered, prioritized and testable requirements, and it is the reference against which the design (D.4.2), the application (D.4.3) and the final report (D.5) will be judged. Every acceptance criterion in Section 6 points to an automated test or a recorded measurement, so a grader can check a requirement without relying on our description of it.

1.2 Problem statement

A ledger database lets an organization prove that stored records were never silently changed. Each change is chained to the previous one by cryptographic hashes, and small summaries of the chain, called digests, are kept somewhere the database administrator cannot rewrite. SQL Server 2022 offers this as a built in feature [5], [6], and Amazon offered a managed document ledger (QLDB) until its end of support in 2025. MongoDB, the document database used in this course, has no ledger at all. An administrator, or anyone holding administrator credentials, can change a balance, delete a record or restore an old backup, and nothing in the database will reveal it afterwards.

The project retrofits tamper evidence onto an unmodified MongoDB 8.0 replica set, using only interfaces MongoDB already provides: multi document transactions and change streams [2], [3]. Two designs are built and compared. **Design S** (synchronous) writes each data change and its ledger entry in the same transaction. **Design A** (asynchronous) is a separate service that reads committed changes from the change stream and seals them into Merkle blocks. Both designs publish block digests outside the database, and one verifier checks either ledger against the digests and against the live data. SQL Server 2022 Ledger serves as the native baseline.

1.3 Objectives

1. Detect, after the fact, any change to protected MongoDB collections that is not explained by the ledger, including changes made by a database administrator.
2. Detect any rewrite, deletion or rollback of the ledger itself once a block digest has been published outside the database.
3. Achieve this without modifying the MongoDB server and at no software cost.
4. Measure the price of each design (throughput, tail latency, storage, and the delay before a change is protected) and compare it with SQL Server 2022 Ledger under the same workloads.
5. Deliver open source code, a repeatable experiment harness and an attack suite that others can run on a laptop.

1.4 Scope and boundaries

In scope	Out of scope
Protected collections in one MongoDB 8.0 replica set (single node in the testbed)	Sharded clusters and multi region deployments
Design S library, Design A service, verifier, digest log with optional Git witness	Preventing tampering; the system detects it, it does not block it
Inserts, updates, replacements and deletes; drops and renames of protected collections	Changes to unprotected collections, indexes, users or server configuration
SQL Server 2022 Ledger baseline with plain and ledger tables	Azure SQL Database and Azure confidential ledger
SmallBank, YCSB workloads A and B, MongoDB sample_analytics dataset	Production deployment, high availability, graphical user interface
Seven scripted insider attacks and their detection matrix	Attacks that compromise both the database and every digest copy

1.5 Definitions

Term	Meaning in this project
Ledger entry	A record of one document change: namespace, document key, operation, hash of the document before and after the change, and the transaction it belongs to.
Block	A sealed group of entries. It stores the Merkle root of its entries (RFC 9162 [4]), counts, a time range and the hash of the previous block.
Digest	The block number and block hash, written to a log outside the database once the block is sealed.
Chain head	In Design S, one of k independent hash chains; the document key decides which head an entry joins.
Change stream	MongoDB's interface for reading committed changes from the replication log (oplog) in order.
Before and after images	Full copies of a document before and after a change, which MongoDB 6.0 and later can attach to change events.
Gap block	A block that records an interval of history Design A could not read, so the loss is explicit rather than silent.
Finding	One problem reported by the verifier, with a kind (for example modified_document), a severity and a detail.

1.6 Assumptions and constraints

- The attacker is an insider with full administrator rights on MongoDB and on the machine that runs it, but cannot rewrite every copy of the digest log.
- The testbed is a single laptop running Docker, with each database limited to two CPUs and 4 GB of memory, and all software is free (MongoDB Community, SQL Server 2022 Developer edition, Python).
- Applications that want Design S protection write through the library; Design A needs no change to applications.
- All data is synthetic: SmallBank and YCSB generate their own records, and sample_analytics is MongoDB's published sample dataset of fictitious customers.
- The work must fit the course schedule: requirements on 23 October, design on 30 October, application, deployment document and user guide on 1 December.

2. Stakeholder Roles

Table 1 lists the people and systems that use or are affected by the ledger, and what each needs from it. The database administrator appears twice on purpose: the same role that runs MongoDB is the threat the ledger is built to expose.

Table 1. Stakeholders and responsibilities

Stakeholder	Who	Responsibilities	What they need from the system
Application developer	Writes the banking or analytics application	Routes protected writes through the Design S library, or leaves writes unchanged under Design A	A small Python API; no change to queries; clear errors on conflicts
Ledger operator	Runs the Design A service and the Design S fold loop	Starts, stops and monitors the processes; keeps the digest log and its witness	Restart without loss; explicit gaps; logs that show progress

Stakeholder	Who	Responsibilities	What they need from the system
Auditor	Internal audit, compliance officer or grader	Runs the verifier, keeps a copy of the digests, investigates findings	One command that answers whether the data can be trusted; a readable JSON report; search over history
Database administrator	Operates MongoDB	Backups, restores, configuration; also the modeled insider threat	Normal administration must still work; misuse must be detectable
External witness	A Git repository outside the team's control	Holds a second, append only copy of the digests	Small text commits; no database access
Course instructor and graders	Dr. Maniatis and teaching assistants	Evaluate the deliverables; run the application	A repository that runs without setup (Codespaces or Docker) and documentation that matches the code
Project team	Ogunjebi: threat model, attacks, verifier. Nelluri: ledger core, Design S, SQL Server. Pratap: Design A, workloads, analysis	Build, test, measure and document; review each other's pull requests	Clear requirements and acceptance tests; a shared plan in Hive

3. Analysis

3.1 Current situation and gap

MongoDB Community records every committed change in the oplog, but the oplog is a capped collection that the server overwrites, and an administrator can restart a node outside the replica set, write directly to its data files or restore an old backup. Auditing is an Enterprise feature and records commands, not proofs of state. Research systems such as LedgerDB [10], GlassDB [11] and VeDB [12] provide verifiable storage, but each is a purpose built engine, and VeriBench [13], which compared verifiable database systems, evaluated only purpose built or rebuilt engines. The gap this project fills is a ledger layer that sits beside an unmodified MongoDB and still gives evidence an auditor can check independently.

3.2 Threat analysis

Table 2 lists the insider actions considered and the requirement that must detect each one. These seven actions form the attack suite (FR25).

Table 2. Insider actions and the requirements that detect them

Insider action	What changes	Detected by
Direct write that skips the Design S library	Data only	FR14 (state scan), FR16 (cross check)
Write while the node runs outside the replica set	Data, with no change event	FR14
Edit of a sealed ledger entry	Ledger	FR12
Consistent rewrite of a whole block and its hash	Ledger	FR13 (digest comparison)
Deletion of the newest blocks (ledger rollback)	Ledger	FR13
Filling the oplog while the Design A service is down	History lost before capture	FR11 (gap block), FR14
Restore of an older backup of data and ledger	Data and ledger together	FR13

3.3 User stories

The functional requirements were derived from the following user stories. Each story is traced to requirements in Section 4.

ID	User story	Requirements
US1	As an auditor, I want one command that tells me whether the protected data still matches its history, so that I can sign off without reading the database by hand.	FR12 to FR17
US2	As an auditor, I want the evidence to survive an administrator who rewrites the database, so that the check does not depend on trusting that person.	FR08, FR09, FR13
US3	As an application developer, I want to protect a transfer with a few lines of code, so that the data change and its ledger entry can never diverge.	FR01 to FR03

ID	User story	Requirements
US4	As an operator of an existing application, I want protection without changing the application, so that legacy writers are covered too.	FR04 to FR06, FR10
US5	As a ledger operator, I want the service to restart safely and to say plainly when it missed history.	FR10, FR11
US6	As an auditor investigating a finding, I want to see every recorded change to one document and prove that a given change is in a sealed block.	FR18 to FR20
US7	As a researcher, I want to measure both designs against SQL Server Ledger under identical workloads.	FR21 to FR24, FR27
US8	As a grader, I want to watch the system detect attacks in a short demonstration.	FR25, FR26

3.4 Prioritization and feasibility

Requirements are prioritized with the MoSCoW method. **Must** requirements are needed to answer the research question and are all scheduled before the D.4 demonstration; **Should** requirements make the system practical for an auditor; **Could** requirements are useful extras. Technically, every Must requirement relies on documented MongoDB 8.0 features. Economically, the cost is zero: all software is free and runs on the team's laptops. On schedule, the ledger core, both designs, the verifier and the attack suite already exist in code, which leaves November for measurement, the remaining Should items and documentation.

4. Functional Requirements

Each requirement states one behavior of the system. Priorities are M (Must), S (Should) and C (Could).

4.1 Change capture

ID	Requirement	Priority
FR01	The system shall write each protected insert, update, replace or delete made through the Design S library together with its ledger entry in one multi document transaction, so that both commit or neither does.	M
FR02	Each ledger entry shall record the namespace, document key, operation, the SHA256 hash of the canonical document before and after the change, and the transaction identifier.	M
FR03	Design S shall link every entry to the previous entry of one of k chain heads, chosen by hashing the document key, with k set at setup.	M
FR04	The Design A service shall read the majority committed change stream of the protected database with before and after images and create an entry for every committed change, including changes that bypass the library.	M
FR05	Design A shall group the events of one multi document transaction, identified by session and transaction number, into one unit within a block.	M
FR06	The setup command shall enable before and after images on the protected collections, create the ledger indexes and write a baseline block for each design that captures every protected document.	M

4.2 Sealing, publication and recovery

ID	Requirement	Priority
FR07	Both designs shall seal pending entries into a block at a configurable interval; the block shall store the RFC 9162 Merkle root of its entries, entry and transaction counts, its time range and the hash of the previous block.	M
FR08	The system shall append the digest of every sealed block to a log file outside the database and flush it to disk before sealing the next block.	M
FR09	The system shall optionally commit and push every digest to a separate Git repository acting as an external witness.	C
FR10	Design A shall store the change stream resume token with each block and, after a restart, continue from it without duplicating or skipping changes.	M
FR11	When the resume point is no longer in the oplog (server errors 286 or 280), Design A shall seal a gap block that records the reason and the restart time, then continue from the current cluster time.	M

4.3 Verification

ID	Requirement	Priority
FR12	The verify command shall check that blocks are numbered without gaps, that each block links to its predecessor, that each block hash is correct and that each Merkle root matches the stored entries.	M
FR13	The verify command shall compare the ledger with the external digest log and report any digest whose block is missing or different.	M
FR14	The verify command shall replay the ledger into the expected state of every protected document, compare it with a full scan, and report missing, modified and unexpected documents.	M
FR15	For Design S, the verify command shall also check every chain head and every link on each head, including entries not yet sealed.	M
FR16	With the cross option, the verify command shall report every change recorded by Design A that has no matching Design S entry.	S
FR17	The verify command shall print a JSON report in which each finding has a kind, a severity (alert, warning or information) and a detail, and shall exit with code 0 when there is no alert and code 2 otherwise.	M
FR18	The prove command shall build and check an RFC 9162 inclusion proof for a chosen ledger entry against the root of its block.	S

4.4 Search and logs

ID	Requirement	Priority
FR19	The history command shall list the ledger entries of one document or one collection, filtered by operation, block range or time range, showing block, operation, hashes and time.	S
FR20	The blocks command shall list blocks with their kind, counts, time range and commit to block lag, and the digests command shall list the external digest log and flag any digest that no longer matches the ledger.	S

4.5 Evaluation tooling

ID	Requirement	Priority
FR21	The system shall provide the SQL Server 2022 baseline: plain and updatable ledger tables holding the same data, digests generated at the same interval as the MongoDB ledgers, and verification with sp_verify_database_ledger.	M
FR22	The system shall provide SmallBank for MongoDB (plain and Design S) and SQL Server with the standard transaction mix [7], and YCSB workloads A and B for MongoDB and SQL Server [8].	M
FR23	The system shall load the MongoDB sample_analytics dataset [9] and run a trading workload over it through either design.	S
FR24	The benchmark harness shall append one row per run to results/summary.csv with throughput, p50, p95 and p99 latency, errors, commit to block lag, verification time and ledger size.	M
FR25	The attack suite shall perform the seven insider actions of Table 2 and append each action and its verification outcome to results/attacks.jsonl.	M
FR26	A demonstration script shall run a clean workload, verify both ledgers and perform three attacks in about two minutes.	S
FR27	The system shall plot throughput, p95 latency and lag from the summary file.	C

5. Nonfunctional Requirements

Quality targets are stated so that they can be measured. Performance targets are goals for the final experiments; the measured values will be reported in D.5 whether or not a target is met.

Table 3. Nonfunctional requirements

ID	Attribute	Requirement	Measure
NFR01	Data integrity	All hashes use SHA256 over a canonical encoding (canonical Extended JSON after a BSON round trip), with separate prefixes for documents, entries, leaves, nodes and blocks.	A change of one field in any sealed entry or block is reported
NFR02	Security, trust	Detection of sealed changes must not depend on trusting the database administrator: digests live outside the database, optionally in a Git witness.	Rollback and rewrite attacks raise an alert

ID	Attribute	Requirement	Measure
NFR03	Security, secrets	No password or key is stored in the repository; credentials come from a local .env file that Git ignores; the Design A service needs only read access to the data and write access to its ledger database.	Repository scan finds no secret
NFR04	Performance, write path	With the Design A service running, SmallBank throughput at 16 clients should be at least 80 percent of the unprotected baseline.	Median of five runs
NFR05	Performance, protection delay	At a 5 second interval, the mean delay from commit to sealed block under Design A should not exceed 6 seconds.	lag_mean_s in summary.csv
NFR06	Performance, audit	Verifying 100,000 SmallBank documents with their ledger should take under 60 seconds on the test laptop.	verify_seconds
NFR07	Reliability	A restart of the service or fold loop loses and duplicates no change; any lost history is recorded as a gap block.	Restart and gap tests pass
NFR08	Consistency	Design S writes use majority write concern and snapshot reads; Design A reads only majority committed changes, so rolled back writes never enter the ledger.	Configuration in code
NFR09	Portability and cost	The system runs on Windows 10 or 11, macOS and Linux through Docker, or in GitHub Codespaces without installation, using only free software.	Clean start in each environment
NFR10	Usability	Every operation is one command with documented options; reports are JSON; every error code in the User Guide has a description and a remedy.	User Guide review
NFR11	Maintainability	Every module and public function carries extended comments; at least 60 automated unit tests run on every push.	CI status on main
NFR12	Reproducibility	Fixed random seeds, container limits of two CPUs and 4 GB, five repeats per configuration, medians with interquartile ranges.	Harness settings
NFR13	Compliance	Only synthetic data is used; software licences are respected (SQL Server Developer edition for development and testing only); University of Windsor academic integrity rules apply.	Dataset sources listed in the repository

6. Acceptance Criteria

A requirement is accepted when its criterion passes. Unit tests run on an in memory stand in for MongoDB; integration tests and the demonstration run on a live MongoDB 8.0 replica set, locally or in the GitHub Actions workflow.

Table 4. Acceptance criteria and evidence

AC	Covers	Condition that must hold	Evidence
AC01	FR01, FR02, FR17	After 300 concurrent transfers through Design S on a live replica set, verify for Design S reports no alert and exits with code 0.	Integration test: concurrent transfers
AC02	FR04, FR05	In the same run, verify for Design A reports no alert, and every entry of a transaction carries the same transaction identifier.	Integration test: concurrent transfers
AC03	FR03, FR15	Changing the chain field of one Design S entry raises a chain hash alert.	Unit test: S chain tampering
AC04	FR14, FR16	A direct update that skips the library raises modified_document for Design S, no alert for Design A, and write_outside_library in the cross check.	Unit and integration tests: bypass
AC05	FR12	Changing the after hash of a sealed entry raises block_content_mismatch.	Unit test: edited sealed entry
AC06	FR13	Rewriting the newest block consistently raises digest_mismatch; deleting it raises digest_without_block.	Unit tests: rewrite and rollback
AC07	FR10, NFR07	A second service run continues after the last sealed block and records each later change exactly once.	Unit test: service resumes
AC08	FR11	When the resume token has left the oplog, a gap block naming error 286 is sealed, verify reports a gap warning, and a change made during the gap is still reported by the scan.	Unit test: history lost
AC09	FR07, FR18	Inclusion proofs verify for every leaf of trees with 1 to 33 leaves, and fail for an altered entry.	Unit tests: inclusion proofs

AC	Covers	Condition that must hold	Evidence
AC10	FR19, FR20	After three updates to one document, history lists exactly those three entries in order; blocks lists every block; digests flags a tampered block.	Unit tests: search commands
AC11	FR21	After a SmallBank run on the ledger table, sp_verify_database_ledger returns 0 against the stored digests.	verify_ok in summary.csv
AC12	FR22 to FR24	Each harness run writes one complete row; the sample_analytics workload runs through both designs and verifies clean.	summary.csv; unit test: analytics
AC13	FR25	All seven attacks appear in attacks.jsonl with the finding that detected them, matching Table 2.	Attack matrix in D.5
AC14	FR26	The demonstration finishes in under three minutes and prints the expected finding for each attack.	Demo output
AC15	NFR04 to NFR06	Measured medians meet the stated targets, or the shortfall is reported and explained.	Final experiments in D.5
AC16	NFR11	The CI workflow passes on the main branch with at least 60 unit tests.	GitHub Actions

7. Revision History

The requirements have evolved with the project, from the preproposal through the approved full proposal to this document. The table records each version, who changed it and who approved it. A requirement that changes after submission keeps its identifier, and every later change will be added here with its approval until the final report (D.5).

Version	Date	Author	Change	Approved by
0.1	Preproposal (D.1)	Project team	Project idea, objectives and scope first stated: a ledger layer for MongoDB compared with SQL Server Ledger	Submitted to the instructor
0.2	4 Oct 2026	Project team	Full proposal (D.2.1): two designs, three workloads, attack suite and evaluation plan defined	Proposal approved by the instructor
0.3	21 Oct 2026	Project team	Requirements rewritten as numbered, prioritized FR01 to FR27 and NFR01 to NFR13; acceptance criteria AC01 to AC16 linked to tests; search requirements (FR19, FR20) and the sample_analytics workload (FR23) added	Team review
1.0	23 Oct 2026	Project team	Reviewed by all three members and submitted as D.4.1	O.O, R.N, Y.P
1.x (planned)	With D.4.2 to D.5	Project team	Any change found during design, testing or the final experiments	To be recorded

References

[1] O. Ogunjebi, R. Nelluri, and Y. Pratap, “A change stream ledger for MongoDB: Tamper evidence without a ledger engine,” COMP.8157 Full Project Proposal (D.2.1), University of Windsor, Oct. 2026.

[2] MongoDB, Inc., “Change streams,” MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/changeStreams/

[3] W. Schultz, T. Avitabile, and A. Cabral, “Tunable consistency in MongoDB,” Proc. VLDB Endow., vol. 12, no. 12, 2019.

[4] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162, IETF, Dec. 2021.

[5] P. Antonopoulos et al., “SQL Ledger: Cryptographically verifiable data in Azure SQL Database,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2021.

[6] Microsoft, “Digest management,” SQL Server documentation, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/security/ledger/ledger-digest-management

[7] M. Alomari, M. Cahill, A. Fekete, and U. Röhm, “The cost of serializability on platforms that use snapshot isolation,” in Proc. IEEE 24th Int. Conf. Data Engineering (ICDE), 2008.

[8] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in Proc. 1st ACM Symp. Cloud Computing (SoCC), 2010.

[9] MongoDB, Inc., “Sample analytics dataset,” MongoDB Atlas documentation. [Online]. Available: https://www.mongodb.com/docs/atlas/sample-data/sample-analytics/

[10] X. Yang et al., “LedgerDB: A centralized ledger database for universal audit and verification,” Proc. VLDB Endow., vol. 13, no. 12, 2020.

[11] C. Yue et al., “GlassDB: An efficient verifiable ledger database system through transparency,” Proc. VLDB Endow., vol. 16, no. 6, 2023.

[12] X. Yang et al., “VeDB: A software and hardware enabled trusted relational database,” Proc. ACM Manag. Data, vol. 1, Art. 194, 2023.

[13] C. Yue, M. Zhang, C. Zhu, G. Chen, D. Loghin, and B. C. Ooi, “VeriBench: Analyzing the performance of database systems with verifiability,” Proc. VLDB Endow., vol. 16, no. 9, 2023.